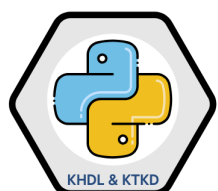


Sắp xếp và biến đổi dữ liệu

Contents

- 9.1. Chỉ số phân cấp
- 9.2. Kết hợp các tập dữ liệu
- 9.3. Định hình lại dữ liệu và thay đổi trực



KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

Trong nhiều ứng dụng, dữ liệu có thể nằm rải rác trên nhiều tệp hoặc cơ sở dữ liệu, hoặc được sắp xếp dưới dạng không thuận tiện cho việc phân tích. Chương này tập trung vào các công cụ giúp kết hợp, nối và sắp xếp lại dữ liệu.

Đầu tiên, chúng ta sẽ làm quen với khái niệm về chỉ số phân cấp (hierarchical indexing) trong `pandas`, một công cụ được sử dụng rộng rãi trong sắp xếp và biến đổi dữ liệu. Sau đó, chương sách sẽ đi sâu vào các thao tác xử lý dữ liệu cụ thể. Bạn đọc có thể xem các ứng dụng thực tế của những công cụ này cả trong các phần sau của cuốn sách

9.1. Chỉ số phân cấp

Chỉ số phân cấp, hay hierarchical indexing, là một tính năng quan trọng của pandas cho phép bạn có nhiều cấp độ chỉ số trên một trục. Một cách khác để hình dung về chỉ số phân cấp là nó cung cấp cho bạn một cách làm việc với dữ liệu nhiều chiều bằng cách thao tác trên các dữ liệu chiều thấp hơn. Hãy bắt đầu với một ví dụ đơn giản: tạo một Series với một danh sách các danh sách hoặc mảng làm chỉ số:

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.uniform(size=9),
                  index=[["a", "a", "a", "b", "b", "c", "c", "d", "d"],
                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

data

```
a  1    0.780314
   2    0.004735
   3    0.853677
b  1    0.128427
   3    0.807250
c  1    0.052421
   2    0.708065
d  2    0.237796
   3    0.148205
dtype: float64
```

```
a  1    0.929616
   2    0.316376
   3    0.183919
b  1    0.204560
   3    0.567725
c  1    0.595545
   2    0.964515
d  2    0.653177
   3    0.748907
dtype: float64
```

Những gì bạn đang thấy là một dạng xem được làm đẹp của một Series với một MultiIndex làm chỉ số hàng. Các “khoảng trống” trong hiển thị chỉ số hàng chỉ mục có nghĩa là sử dụng chỉ số ngay phía trên:

data.index

```
MultiIndex([('a', 1),
            ('a', 2),
            ('a', 3),
            ('b', 1),
            ('b', 3),
            ('c', 1),
            ('c', 2),
            ('d', 2),
            ('d', 3)],
           )
```

```
MultiIndex([('a', 1),
            ('a', 2),
            ('a', 3),
            ('b', 1),
            ('b', 3),
            ('c', 1),
            ('c', 2),
            ('d', 2),
            ('d', 3)],
           )
```

Với một đối tượng có chỉ mục phân cấp, có thể thực hiện thao tác được gọi là *truy cập một phần*, hay partial indexing, cho phép bạn chọn các tập hợp con của dữ liệu một cách ngắn gọn:

```
data["b"]
```

```
1    0.128427
3    0.807250
dtype: float64
```

```
1    0.204560
3    0.567725
dtype: float64
```

```
data["b":"c"]
```

```
b 1    0.128427
   3    0.807250
c 1    0.052421
   2    0.708065
dtype: float64
```

```
b  1    0.204560
   3    0.567725
c  1    0.595545
   2    0.964515
dtype: float64
```

```
data.loc[["b", "d"]]
```

```
b  1    0.128427
   3    0.807250
d  2    0.237796
   3    0.148205
dtype: float64
```

```
b  1    0.204560
   3    0.567725
d  2    0.653177
   3    0.748907
dtype: float64
```

Việc lựa chọn có thể thực hiện từ các chỉ số cấp nhỏ hơn. Ví dụ, chúng ta có thể chọn các giá trị có chỉ số 1 ở cấp chỉ số thứ hai:

```
data.loc[:, 1]
```

```
a    0.780314
b    0.128427
c    0.052421
dtype: float64
```

```
a    0.316376
c    0.964515
d    0.653177
dtype: float64
```

Chỉ số phân cấp đóng một vai trò quan trọng trong việc định hình lại dữ liệu và trong các hoạt động dựa trên nhóm như tạo bảng tổng hợp. Ví dụ, bạn có thể sắp xếp lại dữ liệu này thành một DataFrame bằng phương thức `unstack`:

```
data.unstack()
```

	1	2	3
a	0.780314	0.004735	0.853677
b	0.128427	NaN	0.807250
c	0.052421	0.708065	NaN
d	NaN	0.237796	0.148205

```
a    0.929616    0.316376    0.183919
b    0.204560         NaN    0.567725
c    0.595545    0.964515         NaN
d         NaN    0.653177    0.748907
```

Thao tác ngược lại của `unstack` là `stack`:

```
data.unstack().stack()
```

```
a  1    0.780314
   2    0.004735
   3    0.853677
b  1    0.128427
   3    0.807250
c  1    0.052421
   2    0.708065
d  2    0.237796
   3    0.148205
dtype: float64
```

```
a  1    0.929616
   2    0.316376
   3    0.183919
b  1    0.204560
   3    0.567725
c  1    0.595545
   2    0.964515
d  2    0.653177
   3    0.748907
dtype: float64
```

`stack` và `unstack` sẽ được khám phá chi tiết hơn sau trong phần sắp xếp và chuyển đổi dữ liệu

Với một DataFrame, cả hàng và cột đều có thể có chỉ số phân cấp:

```
frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
                      index=[["a", "a", "b", "b"], [1, 2, 1, 2]],
                      columns=[["Ohio", "Ohio", "Colorado",
                                ["Green", "Red", "Green"]])
```

frame

		Ohio		Colorado
		Green	Red	Green
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
      Ohio  Colorado
a 1  Green  Red    Green
   1      0   1      2
   2      3   4      5
b 1  Green  Red    Green
   1      6   7      8
   2      9  10     11
```

Các cấp phân cấp có thể được đặt tên dưới dạng chuỗi của Python.

```
frame.index.names = ["key1", "key2"]
```

```
frame.columns.names = ["state", "color"]
```

frame

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```

state    Ohio      Colorado
color   Green Red    Green
key1 key2
a      1      0      1      2
      2      3      4      5
b      1      6      7      8
      2      9     10     11

```

Những tên này thay thế thuộc tính `name`, vốn chỉ được sử dụng với các chỉ số chỉ có một cấp.

Lưu ý

Hãy cẩn thận lưu ý rằng tên chỉ số "state" và "color" là một phần của tên hàng và có thể được gọi bằng thuộc tính `index`.

Bạn có thể xem một chỉ số có bao nhiêu cấp bằng cách sử dụng thuộc tính `nlevels`:

```
frame.index.nlevels
```

```
2
```

```
2
```

Với việc truy cập một phần cột, bạn cũng có thể chọn các nhóm cột tương tự:

```
frame["Ohio"]
```

	color	Green	Red
key1	key2		
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

```

color      Green  Red
key1 key2
a    1      0    1
     2      3    4
b    1      6    7
     2      9   10

```

9.1.1. Sắp xếp lại và sắp xếp các cấp

Đôi khi chúng ta có thể cần sắp xếp lại thứ tự các cấp của chỉ số hoặc sắp xếp dữ liệu theo giá trị ở một cấp cụ thể. Phương thức `swaplevel` nhận hai số hoặc tên cấp và trả về một đối tượng mới với các cấp được hoán đổi nhưng dữ liệu không thay đổi:

```
frame.swaplevel("key1", "key2")
```

	state	Ohio		Colorado
	color	Green	Red	Green
key2	key1			
1	a	0	1	2
2	a	3	4	5
1	b	6	7	8
2	b	9	10	11


```

state  Ohio      Colorado
color  Green Red   Green
key2 key1
1      a         0   1       2
2      a         3   4       5
1      b         6   7       8
2      b         9  10      11

```

`sort_index` theo mặc định sắp xếp dữ liệu theo thứ tự từ điển sử dụng tất cả các cấp chỉ số, nhưng bạn có thể chọn chỉ sử dụng một cấp hoặc một tập hợp con các cấp để sắp xếp bằng cách truyền tham số `level`. Ví dụ:

```
frame.sort_index(level=1)
```

	state	Ohio		Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
b	1	6	7	8
a	2	3	4	5
b	2	9	10	11

```

state  Ohio      Colorado
color  Green Red   Green
key1 key2
a      1         0   1       2
b      1         6   7       8
a      2         3   4       5
b      2         9  10      11

```

```
frame.swaplevel(0, 1).sort_index(level=0)
```

	state	Ohio		Colorado
	color	Green	Red	Green
key2	key1			
1	a	0	1	2
	b	6	7	8
2	a	3	4	5
	b	9	10	11

```

state    Ohio    Colorado
color   Green Red    Green
key2 key1
1      a      0      1      2
      b      6      7      8
2      a      3      4      5
      b      9     10     11

```

Lưu ý

Hiệu suất lựa chọn dữ liệu tốt hơn nhiều trên các đối tượng có chỉ số phân cấp nếu chỉ số được sắp xếp theo thứ tự từ điển bắt đầu từ cấp ngoài cùng—nghĩa là, kết quả của việc gọi

`sort_index(level=0)` hoặc `sort_index()`.

9.1.2. Thống kê tóm tắt theo cấp

Nhiều thống kê mô tả và tóm tắt trên `DataFrame` và `Series` có một tùy chọn `level` trong đó bạn có thể chỉ định cấp bạn muốn tổng hợp theo trên một hàng hay một cột. Hãy xem xét `DataFrame` ở trên; chúng ta có thể tổng hợp theo cấp trên hàng hoặc cột, như sau:

```
frame.groupby(level="key2").sum()
```

state	Ohio		Colorado
color	Green	Red	Green
key2			
1	6	8	10
2	12	14	16

```
state  Ohio      Colorado
color  Green  Red      Green
key2
1         6    8         10
2        12   14        16
```

```
frame.groupby(level="color", axis="columns").sum()
```

	color	Green	Red
key1	key2		
a	1	2	1
	2	8	4
b	1	14	7
	2	20	10

```
color      Green  Red
key1 key2
a      1         2    1
      2         8    4
b      1        14    7
      2        20   10
```

Chúng ta sẽ thảo luận chi tiết hơn về `groupby` sau trong phần tổng hợp dữ liệu.

9.1.3. Tạo chỉ số bằng các cột của dataframe

Trong nhiều trường hợp, chúng ta có thể sử dụng một hoặc nhiều cột từ DataFrame làm chỉ số và ngược lại, bạn có thể muốn chuyển chỉ số vào các cột của DataFrame. Ví dụ:

```
frame = pd.DataFrame({"a": range(7), "b": range(7, 0, -1),  
                      "c": ["one", "one", "one", "two", "two",  
                           "two", "two"],  
                      "d": [0, 1, 2, 0, 1, 2, 3]})
```

frame

	a	b	c	d
0	0	7	one	0
1	1	6	one	1
2	2	5	one	2
3	3	4	two	0
4	4	3	two	1
5	5	2	two	2
6	6	1	two	3

```
   a  b  c  d  
0  0  7 one  0  
1  1  6 one  1  
2  2  5 one  2  
3  3  4 two  0  
4  4  3 two  1  
5  5  2 two  2  
6  6  1 two  3
```

Hàm `set_index` của DataFrame sẽ tạo một DataFrame mới sử dụng một hoặc nhiều cột làm chỉ số:

```
frame2 = frame.set_index(["c", "d"])
```

frame2

		a	b
c	d		
one	0	0	7
	1	1	6
	2	2	5
two	0	3	4
	1	4	3
	2	5	2
	3	6	1

```

      a  b
c  d
one 0  0  7
     1  1  6
     2  2  5
two  0  3  4
     1  4  3
     2  5  2
     3  6  1

```

Theo mặc định, các cột được loại bỏ khỏi DataFrame, mặc dù chúng ta có thể giữ chúng lại bằng cách truyền `drop=False` cho `set_index`:

```
frame.set_index(["c", "d"], drop=False)
```

		a	b	c	d
c	d				
one	0	0	7	one	0
	1	1	6	one	1
	2	2	5	one	2
two	0	3	4	two	0
	1	4	3	two	1
	2	5	2	two	2
	3	6	1	two	3

		a	b	c	d
c	d				
one	0	0	7	one	0
	1	1	6	one	1
	2	2	5	one	2
two	0	3	4	two	0
	1	4	3	two	1
	2	5	2	two	2
	3	6	1	two	3

Phương thức `reset_index`, mặt khác, thực hiện ngược lại với `set_index`; các phân cấp chỉ số được chuyển vào các cột:

```
frame2.reset_index()
```

	c	d	a	b
0	one	0	0	7
1	one	1	1	6
2	one	2	2	5
3	two	0	3	4
4	two	1	4	3
5	two	2	5	2
6	two	3	6	1

```

      c  d  a  b
0  one  0  0  7
1  one  1  1  6
2  one  2  2  5
3  two  0  3  4
4  two  1  4  3
5  two  2  5  2
6  two  3  6  1

```

9.2. Kết hợp các tập dữ liệu

Trong quá trình phân tích và xử lý dữ liệu, nhu cầu kết hợp nhiều nguồn dữ liệu khác nhau là rất phổ biến. Thư viện **pandas** cung cấp một số phương pháp linh hoạt và hiệu quả để kết hợp dữ liệu được lưu trữ trong các đối tượng `DataFrame`. Cụ thể, ba phương pháp thường được sử dụng bao gồm:

- `pandas.merge`
Phương pháp này cho phép kết nối các hàng giữa hai hoặc nhiều `DataFrame` dựa trên một hoặc nhiều khóa chung. Cách tiếp cận này tương tự với các thao tác *join* trong ngôn ngữ truy vấn SQL và thường được sử dụng trong các hệ quản trị cơ sở dữ liệu quan hệ.
- `pandas.concat` Được sử dụng để ghép nối các đối tượng dọc theo một trục xác định (theo hàng hoặc theo cột). Phương pháp này thích hợp trong các tình huống cần kết hợp dữ liệu có cùng cấu trúc.
- `combine_first` Dùng để kết hợp hai đối tượng có cấu trúc tương tự, trong đó các giá trị bị thiếu (*missing values*) trong một đối tượng sẽ được điền bởi các giá trị tương ứng từ đối

tượng còn lại. Đây là phương pháp hữu ích trong việc làm sạch và bổ sung dữ liệu.

Trong các phần tiếp theo, chúng ta sẽ trình bày chi tiết từng phương pháp trên kèm theo ví dụ minh họa cụ thể. Những phương pháp này cũng sẽ được sử dụng xuyên suốt các chương còn lại của cuốn sách để giải quyết các tình huống thực tiễn trong phân tích dữ liệu.

9.2.1. Các phép gộp và nối dữ liệu

Các hoạt động **gộp (merge)** hoặc **nối (join)** cho phép kết hợp nhiều tập dữ liệu lại với nhau bằng cách liên kết các hàng dựa trên một hoặc nhiều khóa chung. Đây là các thao tác cốt lõi trong phân tích dữ liệu, đặc biệt quan trọng trong ngữ cảnh của các hệ quản trị cơ sở dữ liệu quan hệ, chẳng hạn như các hệ thống dựa trên SQL.

Trong thư viện **pandas**, hàm `pandas.merge` là công cụ chính để thực hiện các thao tác gộp dữ liệu, triển khai các thuật toán tương tự như các phép *join* trong SQL, bao gồm `inner`, `outer`, `left`, và `right join`.

Hãy bắt đầu với một ví dụ đơn giản:

```
df1 = pd.DataFrame({"key": ["b", "b", "a", "c", "a", "a", "b"],  
                    "data1": pd.Series(range(7), dtype="Int64")})
```

```
df2 = pd.DataFrame({"key": ["a", "b", "d"],  
                    "data2": pd.Series(range(3), dtype="Int64")})
```

```
df1
```


	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

```

key  data1
0    b     0
1    b     1
2    a     2
3    c     3
4    a     4
5    a     5
6    b     6

```

df2

	key	data2
0	a	0
1	b	1
2	d	2

```

key  data2
0    a     0
1    b     1
2    d     2

```

Đây là một ví dụ minh họa cho phép nối **hiều-một** (*many-to-one*), trong đó dữ liệu trong `df1` chứa nhiều hàng có cùng tên, chẳng hạn như `'a'` và `'b'`, trong khi `df2` chỉ chứa một hàng tương ứng cho mỗi giá trị trong cột `key`.

Khi gọi hàm `pandas.merge` để kết hợp hai bảng dữ liệu này, các hàng trong `df1` sẽ được ghép nối với các hàng phù hợp trong `df2` dựa trên giá trị khóa. Kết quả thu được là một dữ liệu mới, trong đó mỗi hàng của `df1` được bổ sung thông tin từ `df2` theo cách tương ứng với cấu trúc nhiều-một.

Cụ thể, ta thực hiện như sau:

```
pd.merge(df1, df2)
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	a	2	0
3	a	4	0
4	a	5	0
5	b	6	1

```
key  data1  data2
0    b      0      1
1    b      1      1
2    b      6      1
3    a      2      0
4    a      4      0
5    a      5      0
```

Trong câu lệnh ở trên, chúng ta không chỉ định cột nào để join. Nếu thông tin đó không được chỉ định, `pandas.merge` sử dụng các tên cột giống nhau làm khóa. Tuy nhiên, chúng tôi khuyến khích bạn đọc nên chỉ định rõ ràng tên cột dùng để join.

```
pd.merge(df1, df2, on="key")
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	a	2	0
3	a	4	0
4	a	5	0
5	b	6	1

```

key  data1  data2
0    b      0      1
1    b      1      1
2    b      6      1
3    a      2      0
4    a      4      0
5    a      5      0

```

Nói chung, thứ tự của các cột trong dữ liệu đầu ra từ `pandas.merge` là không xác định. Nếu tên cột khác nhau trong mỗi đối tượng, chúng ta có thể chỉ định. Theo mặc định, `pandas.merge` thực hiện phép join trong (`inner`). Các tùy chọn khả thi khác là `"left"`, `"right"`, và `"outer"`. Phép join ngoài (`outer join`) lấy hợp của các khóa, kết hợp hiệu ứng của việc áp dụng cả phép join trái và phải:

```
pd.merge(df1, df2, how="outer")
```

	key	data1	data2
0	a	2	0
1	a	4	0
2	a	5	0
3	b	0	1
4	b	1	1
5	b	6	1
6	c	3	<NA>
7	d	<NA>	2

```

key data1 data2
0    b     0     1
1    b     1     1
2    b     6     1
3    a     2     0
4    a     4     0
5    a     5     0
6    c     3  <NA>
7    d  <NA>     2

```

Trong phép join ngoài, các hàng từ đối tượng DataFrame bên trái hoặc bên phải không khớp khóa trong DataFrame kia sẽ xuất hiện với giá trị **NA** trong các cột của DataFrame kia cho các hàng không khớp.

Xem Bảng ... để biết tóm tắt các tùy chọn cho **how**.

Bảng ... Các loại join khác nhau với đối số **how**

Tùy chọn	Hành vi
<code>how="inner"</code>	Chỉ sử dụng các tổ hợp khóa được quan sát thấy trong cả hai bảng
<code>how="left"</code>	Sử dụng tất cả các tổ hợp khóa được tìm thấy trong bảng bên trái
<code>how="right"</code>	Sử dụng tất cả các tổ hợp khóa được tìm thấy trong bảng bên phải
<code>how="outer"</code>	Sử dụng tất cả các tổ hợp khóa được quan sát thấy trong cả hai bảng cùng nhau

Các phép gộp nhiều-nhiều (many-to-many) tạo thành tích Descartes (Cartesian product) của các khóa khớp. Đây là một ví dụ:

```
df1 = pd.DataFrame({"key": ["b", "b", "a", "c", "a", "b"],
                    "data1": pd.Series(range(6), dtype="Int64")})
```

```
df2 = pd.DataFrame({"key": ["a", "b", "a", "b", "d"],
                    "data2": pd.Series(range(5), dtype="Int64")})
```

```
df1
```

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

```
   key  data1
0    b      0
1    b      1
2    a      2
3    c      3
4    a      4
5    b      5
```

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

```
   key  data2
0    a      0
1    b      1
2    a      2
3    b      3
4    d      4
```

```
pd.merge(df1, df2, on="key", how="left")
```

	key	data1	data2
0	b	0	1
1	b	0	3
2	b	1	1
3	b	1	3
4	a	2	0
5	a	2	2
6	c	3	<NA>
7	a	4	0
8	a	4	2
9	b	5	1
10	b	5	3

```

key data1 data2
0    b     0     1
1    b     0     3
2    b     1     1
3    b     1     3
4    a     2     0
5    a     2     2
6    c     3  <NA>
7    a     4     0
8    a     4     2
9    b     5     1
10   b     5     3

```

Vì có ba hàng "b" trong DataFrame bên trái và hai hàng trong DataFrame bên phải, nên có sáu hàng "b" trong kết quả. Phương thức join được truyền cho đối số từ khóa `how` chỉ ảnh hưởng đến các giá trị khóa riêng biệt xuất hiện trong kết quả:

```
pd.merge(df1, df2, how="inner")
```

	key	data1	data2
0	b	0	1
1	b	0	3
2	b	1	1
3	b	1	3
4	a	2	0
5	a	2	2
6	a	4	0
7	a	4	2
8	b	5	1
9	b	5	3

	key	data1	data2
0	b	0	1
1	b	0	3
2	b	1	1
3	b	1	3
4	b	5	1
5	b	5	3
6	a	2	0
7	a	2	2
8	a	4	0
9	a	4	2

Để gộp với nhiều khóa, hãy truyền một danh sách tên cột:

```
left = pd.DataFrame({"key1": ["foo", "foo", "bar"],
                      "key2": ["one", "two", "one"],
                      "lval": pd.Series([1, 2, 3], dtype='Int64')})
```

```
right = pd.DataFrame({"key1": ["foo", "foo", "bar", "bar"],
                      "key2": ["one", "one", "one", "two"],
                      "rval": pd.Series([4, 5, 6, 7], dtype='Int64')})
```

```
pd.merge(left, right, on=["key1", "key2"], how="outer")
```


	key1	key2	lval	rval
0	bar	one	3	6
1	bar	two	<NA>	7
2	foo	one	1	4
3	foo	one	1	5
4	foo	two	2	<NA>

```

key1 key2 lval rval
0  foo  one    1    4
1  foo  one    1    5
2  foo  two    2  <NA>
3  bar  one    3    6
4  bar  two  <NA>    7

```

Để xác định tổ hợp khóa nào sẽ xuất hiện trong kết quả tùy thuộc vào lựa chọn phương thức gộp, hãy coi nhiều khóa như tạo thành một mảng các tuple được sử dụng làm một khóa join duy nhất.

Lưu ý

Khi bạn join cột với cột, các chỉ số trên các đối tượng DataFrame được truyền vào sẽ bị loại bỏ. Nếu bạn cần giữ lại các giá trị chỉ số, bạn có thể sử dụng `reset_index` để nối chỉ mục vào các cột.

Một vấn đề cũng cần xem xét trong các hoạt động kết hợp dữ liệu là việc xử lý các tên cột giống nhau trong 2 dữ liệu. Ví dụ:

```
pd.merge(left, right, on="key1")
```

	key1	key2_x	lval	key2_y	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

```

key1 key2_x lval key2_y rval
0  foo    one    1    one    4
1  foo    one    1    one    5
2  foo    two    2    one    4
3  foo    two    2    one    5
4  bar    one    3    one    6
5  bar    one    3    two    7

```

Bạn đọc có thể sử dụng tham số `suffixes` để chỉ định các chuỗi được nối vào các tên cột chéo trong các đối tượng DataFrame bên trái và bên phải:

```
pd.merge(left, right, on="key1", suffixes=("_left", "_right"))
```

	key1	key2_left	lval	key2_right	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

	key1	key2_left	lval	key2_right	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

Xem Bảng ... để tham khảo các tham số của hàm `pandas.merge`. Phần tiếp theo bao gồm việc join bằng chỉ số hàng của DataFrame.

Bảng: Các tham số của hàm `pandas.merge`

Đối số	Mô tả
<code>left</code>	DataFrame được gộp ở phía bên trái.
<code>right</code>	DataFrame được gộp ở phía bên phải.
<code>how</code>	Loại join để áp dụng: một trong số <code>"inner"</code> , <code>"outer"</code> , <code>"left"</code> , hoặc <code>"right"</code> ; mặc định là <code>"inner"</code> .
<code>on</code>	Tên cột để join. Phải được tìm thấy trong cả hai đối tượng DataFrame. Nếu không được chỉ định và không có khóa join nào khác được cung cấp, sẽ sử dụng giao điểm của các tên cột trong <code>left</code> và <code>right</code> làm khóa join.
<code>left_on</code>	Các cột trong DataFrame <code>left</code> để sử dụng làm khóa join. Có thể là một tên cột duy nhất hoặc một danh sách các tên cột.
<code>right_on</code>	Tương tự như <code>left_on</code> cho DataFrame <code>right</code> .
<code>left_index</code>	Sử dụng chỉ mục hàng trong <code>left</code> làm khóa join của nó (hoặc các khóa, nếu là <code>MultiIndex</code>).
<code>right_index</code>	Tương tự như <code>left_index</code> .
<code>sort</code>	Sắp xếp dữ liệu đã gộp theo thứ tự từ điển bằng các khóa join; <code>False</code> theo mặc định.
<code>suffixes</code>	Tuple các giá trị chuỗi để nối vào tên cột trong trường hợp chồng chéo; mặc định là <code>("_x", "_y")</code> (ví dụ: nếu "data" có trong cả hai đối tượng DataFrame, sẽ xuất hiện dưới dạng "data_x" và "data_y" trong kết quả).
<code>copy</code>	Nếu <code>False</code> , tránh sao chép dữ liệu vào cấu trúc dữ liệu kết quả trong một số trường hợp ngoại lệ; theo mặc định luôn sao chép.
<code>validate</code>	Xác minh xem phép gộp có thuộc loại được chỉ định hay không, cho dù là một-một, một-nhiều, hay nhiều-nhiều. Xem docstring để biết chi tiết đầy đủ về các tùy chọn.
<code>indicator</code>	Thêm một cột đặc biệt <code>_merge</code> chỉ ra nguồn của mỗi hàng; các giá trị sẽ là <code>"left_only"</code> , <code>"right_only"</code> , hoặc <code>"both"</code> dựa trên nguồn gốc của dữ liệu được join trong mỗi hàng.

9.2.2. Kết hợp dữ liệu dựa trên chỉ số

Trong một số trường hợp, các khóa trong một DataFrame sẽ được tìm thấy trong chỉ số hàng. Trong trường hợp này, bạn có thể truyền `left_index=True` hoặc `right_index=True` (hoặc cả hai) để chỉ ra rằng chỉ số được sử dụng làm khóa:

```
left1 = pd.DataFrame({"key": ["a", "b", "a", "a", "b", "c"],  
                      "value": pd.Series(range(6), dtype="Int64")})
```

```
right1 = pd.DataFrame({"group_val": [3.5, 7]}, index=["a", "b"])
```

left1

	key	value
0	a	0
1	b	1
2	a	2
3	a	3
4	b	4
5	c	5

```
   key  value  
0    a      0  
1    b      1  
2    a      2  
3    a      3  
4    b      4  
5    c      5
```

right1

	group_val
a	3.5
b	7.0

```

group_val
a      3.5
b      7.0

```

```
pd.merge(left1, right1, left_on="key", right_index=True)
```

	key	value	group_val
0	a	0	3.5
1	b	1	7.0
2	a	2	3.5
3	a	3	3.5
4	b	4	7.0

```

   key  value  group_val
0    a      0        3.5
2    a      2        3.5
3    a      3        3.5
1    b      1        7.0
4    b      4        7.0

```

Lưu ý

Nếu bạn xem xét kỹ ví dụ trên, bạn sẽ thấy rằng các giá trị chỉ số cho `left1` đã được giữ lại, trong khi ở các ví dụ khác ở trên, các chỉ số của các đối tượng DataFrame đầu vào bị loại bỏ. Bởi vì chỉ số của `right1` là duy nhất, phép gộp "nhiều-một" này (với phương thức `how="inner"` mặc định) có thể giữ lại các giá trị chỉ số từ `left1` tương ứng với các hàng trong đầu ra.

Vì phương thức gộp mặc định là lấy phần giao của các khóa join, bạn có thể sử dụng phép join ngoài nếu muốn lấy phần hợp của các khóa join:

```
pd.merge(left1, right1, left_on="key", right_index=True, how="outer")
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0
5	c	5	NaN

```
   key  value  group_val
0    a      0      3.5
2    a      2      3.5
3    a      3      3.5
1    b      1      7.0
4    b      4      7.0
5    c      5      NaN
```

9.2.3. Nối dữ liệu theo một trục

Một loại hoạt động kết hợp dữ liệu khác được gọi thay thế cho nhau là nối dữ liệu. Hàm `concatenate` của NumPy có thể thực hiện điều này với các mảng NumPy:

```
arr = np.arange(12).reshape((3, 4))
```

```
arr
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
np.concatenate([arr, arr], axis=1)
```

```
array([[ 0,  1,  2,  3,  0,  1,  2,  3],
       [ 4,  5,  6,  7,  4,  5,  6,  7],
       [ 8,  9, 10, 11,  8,  9, 10, 11]])
```

```
array([[ 0,  1,  2,  3,  0,  1,  2,  3],
       [ 4,  5,  6,  7,  4,  5,  6,  7],
       [ 8,  9, 10, 11,  8,  9, 10, 11]])
```

Đối với các đối tượng `pandas` như `Series` và `DataFrame`, việc mỗi trục đều được gán nhãn cho phép chúng ta mở rộng khái niệm nối mảng với mảng theo hướng tổng quát và linh hoạt hơn. Khi thao tác với các đối tượng này, một số vấn đề bổ sung cần được xem xét nhằm đảm bảo tính nhất quán và toàn vẹn của dữ liệu sau khi thực hiện phép nối:

- **Thứ nhất**, khi các đối tượng có hệ thống chỉ số khác nhau trên trục được nối, cần quyết định liệu có nên giữ lại tất cả các chỉ số hay chỉ sử dụng phần giao nhau của chúng.
- **Thứ hai**, liệu các dữ liệu đầu vào có cần được bảo toàn dưới dạng riêng biệt trong đối tượng kết quả hay không, tức là có nên tạo phân vùng rõ ràng giữa các phần được nối?
- **Thứ ba**, trục được sử dụng để nối dữ liệu có chứa thông tin có ý nghĩa cần được bảo toàn không? Trong nhiều trường hợp thực hành, các chỉ số dạng số nguyên mặc định của `DataFrame` không mang thông tin và nên được loại bỏ trong quá trình kết hợp.

Hàm `concat` trong thư viện `pandas` cung cấp một cơ chế thống nhất và linh hoạt để giải quyết các câu hỏi nêu trên. Trong các phần tiếp theo, chúng ta sẽ minh họa cách thức hoạt động của `concat` thông qua các ví dụ cụ thể.

Giả sử chúng ta có ba đối tượng `Series` với các chỉ số không chồng chéo:

```
s1 = pd.Series([0, 1], index=["a", "b"], dtype="Int64")
```

```
s2 = pd.Series([2, 3, 4], index=["c", "d", "e"], dtype="Int64")
```



```
s3 = pd.Series([5, 6], index=["f", "g"], dtype="Int64")
```

Gọi `pandas.concat` với các đối tượng này trong một danh sách sẽ gắn kết các giá trị và chỉ số lại với nhau:

```
s1
```

```
a    0
b    1
dtype: Int64
```

```
a    0
b    1
dtype: Int64
```

```
s2
```

```
c    2
d    3
e    4
dtype: Int64
```

```
c    2
d    3
e    4
dtype: Int64
```

```
s3
```

```
f    5
g    6
dtype: Int64
```

```
f    5
g    6
dtype: Int64
```

```
pd.concat([s1, s2, s3])
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
dtype: Int64
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
dtype: Int64
```

Theo mặc định, `pandas.concat` hoạt động dọc theo hàng, hay `axis="index"`, để tạo ra một `Series` khác. Nếu bạn truyền `axis="columns"`, kết quả sẽ là một DataFrame:

```
pd.concat([s1, s2, s3], axis="columns")
```

	0	1	2
a	0	<NA>	<NA>
b	1	<NA>	<NA>
c	<NA>	2	<NA>
d	<NA>	3	<NA>
e	<NA>	4	<NA>
f	<NA>	<NA>	5
g	<NA>	<NA>	6

	0	1	2
a	0	<NA>	<NA>
b	1	<NA>	<NA>
c	<NA>	2	<NA>
d	<NA>	3	<NA>
e	<NA>	4	<NA>
f	<NA>	<NA>	5
g	<NA>	<NA>	6

Trong trường hợp này, bạn đọc có thể thấy kết quả là hợp, hay phép join ngoài của các chỉ số. Thay vào đó, bạn có thể lấy phần giao của chỉ số bằng cách truyền `join="inner"`:

```
s4 = pd.concat([s1, s3])
```

```
s4
```

```
a    0
b    1
f    5
g    6
dtype: Int64
```

```
a    0
b    1
f    5
g    6
dtype: Int64
```

```
pd.concat([s1, s4], axis="columns")
```

	0	1
a	0	0
b	1	1
f	<NA>	5
g	<NA>	6

```

      0  1
a  0  0
b  1  1
f <NA> 5
g <NA> 6

```

```
pd.concat([s1, s4], axis="columns", join="inner")
```

	0	1
a	0	0
b	1	1

```

      0  1
a  0  0
b  1  1

```

Trong ví dụ cuối cùng này, các chỉ số "f" và "g" đã biến mất do tùy chọn `join="inner"`.

Trong trường hợp kết hợp Series dọc theo `axis="columns"`, các `keys` trở thành tiêu đề cột của DataFrame:

```
pd.concat([s1, s2, s3], axis="columns", keys=["one", "two", "three"])
```

	one	two	three
a	0	<NA>	<NA>
b	1	<NA>	<NA>
c	<NA>	2	<NA>
d	<NA>	3	<NA>
e	<NA>	4	<NA>
f	<NA>	<NA>	5
g	<NA>	<NA>	6

	one	two	three
a	0	<NA>	<NA>
b	1	<NA>	<NA>
c	<NA>	2	<NA>
d	<NA>	3	<NA>
e	<NA>	4	<NA>
f	<NA>	<NA>	5
g	<NA>	<NA>	6

Logic tương tự cho các đối tượng DataFrame:

```
df1 = pd.DataFrame(np.arange(6).reshape(3, 2), index=["a", "b", "c"],
                    columns=["one", "two"])
```

```
df2 = pd.DataFrame(5 + np.arange(4).reshape(2, 2), index=["a", "c"],
                    columns=["three", "four"])
```

df1

	one	two
a	0	1
b	2	3
c	4	5

	one	two
a	0	1
b	2	3
c	4	5

df2

	three	four
a	5	6
c	7	8

	three	four
a	5	6
c	7	8

```
pd.concat([df1, df2], axis="columns", keys=["level1", "level2"])
```

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

Ở đây, tham số `keys` được sử dụng để tạo một chỉ số phân cấp nơi cấp đầu tiên có thể được sử dụng để xác định từng đối tượng DataFrame được nối.

Nếu bạn truyền một từ điển các đối tượng thay vì một danh sách, các khóa của từ điển sẽ được sử dụng cho tùy chọn `keys`:

```
pd.concat({"level1": df1, "level2": df2}, axis="columns")
```

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

Có các tham số bổ sung chi phối cách tạo chỉ số phân cấp. Ví dụ, chúng ta có thể đặt tên cho các cấp trực được tạo bằng tham số `names`:

```
pd.concat([df1, df2], axis="columns", keys=["level1", "level2"], names=["upper", "lower"])
```

	upper	level1		level2	
	lower	one	two	three	four
	a	0	1	5.0	6.0
	b	2	3	NaN	NaN
	c	4	5	7.0	8.0

	upper	level1		level2	
	lower	one	two	three	four
a		0	1	5.0	6.0
b		2	3	NaN	NaN
c		4	5	7.0	8.0

Một xem xét cuối cùng liên quan đến các DataFrame trong đó chỉ số hàng không chứa bất kỳ dữ liệu liên quan nào:

```
df1 = pd.DataFrame(np.random.standard_normal((3, 4)),
                    columns=["a", "b", "c", "d"])
```

```
df2 = pd.DataFrame(np.random.standard_normal((2, 3)),
                    columns=["b", "d", "a"])
```

```
df1
```

	a	b	c	d
0	-0.706160	-0.763535	-0.690340	1.760008
1	1.822410	0.450920	-0.890852	0.167614
2	0.957429	-0.204106	-0.793911	1.054604

```
      a      b      c      d
0  1.248804  0.774191 -0.319657 -0.624964
1  1.078814  0.544647  0.855588  1.343268
2 -0.267175  1.793095 -0.652929 -1.886837
```

df2

	b	d	a
0	0.459392	-0.309329	-1.965219
1	-0.784532	1.345855	0.659401

```
      b      d      a
0  1.059626  0.644448 -0.007799
1 -0.449204  2.448963  0.667226
```

Trong trường hợp này, bạn có thể truyền `ignore_index=True`, thao tác này sẽ loại bỏ các chỉ số từ mỗi DataFrame và chỉ nối dữ liệu trong các cột, gán một chỉ số mặc định mới:

```
pd.concat([df1, df2], ignore_index=True)
```

	a	b	c	d
0	-0.706160	-0.763535	-0.690340	1.760008
1	1.822410	0.450920	-0.890852	0.167614
2	0.957429	-0.204106	-0.793911	1.054604
3	-1.965219	0.459392	NaN	-0.309329
4	0.659401	-0.784532	NaN	1.345855

	a	b	c	d
0	1.248804	0.774191	-0.319657	-0.624964
1	1.078814	0.544647	0.855588	1.343268
2	-0.267175	1.793095	-0.652929	-1.886837
3	-0.007799	1.059626	NaN	0.644448
4	0.667226	-0.449204	NaN	2.448963

Bảng ... mô tả các tham số của hàm `pandas.concat`.

Bảng ... các tham số của hàm `pandas.concat`

Đối số	Mô tả
<code>objs</code>	Danh sách hoặc từ điển các đối tượng pandas cần nối; đây là đối số bắt buộc duy nhất.
<code>axis</code>	Trục để nối dọc theo; mặc định là nối dọc theo hàng (<code>axis="index"</code>).
<code>join</code>	Hoặc <code>"inner"</code> hoặc <code>"outer"</code> (<code>"outer"</code> theo mặc định); liệu có lấy giao điểm (inner) hay hợp (outer) các chỉ số dọc theo các trục khác.
<code>keys</code>	Các giá trị để liên kết với các đối tượng đang được nối, tạo thành một chỉ số phân cấp dọc theo trục nối; có thể là một danh sách hoặc mảng các giá trị tùy ý, một mảng các tuple, hoặc một danh sách các mảng (nếu nhiều mảng cấp được truyền vào <code>levels</code>).
<code>names</code>	Tên cho các cấp phân cấp được tạo nếu <code>keys</code> và/hoặc <code>levels</code> được truyền.
<code>verify_integrity</code>	Kiểm tra trục mới trong đối tượng được nối xem có trùng lặp không và đưa ra một ngoại lệ nếu có; theo mặc định (<code>False</code>) cho phép trùng lặp.
<code>ignore_index</code>	Không bảo toàn các chỉ số dọc theo trục nối, thay vào đó tạo ra một chỉ số <code>range(total_length)</code> mới.

9.3. Định hình lại dữ liệu và thay đổi trục

Có một số hoạt động cơ bản để sắp xếp lại dữ liệu dạng bảng, các hoạt động này bao gồm định hình lại dữ liệu, hay reshape, hoặc và thay đổi trục hay còn gọi là pivot.

9.3.1. Định hình lại với chỉ số phân cấp

Chỉ số phân cấp cung cấp một cách nhất quán để sắp xếp lại dữ liệu trong một `DataFrame`. Có hai thao tác chính khi định hình lại là:

- `stack`: thao tác này "xoay" hoặc chuyển các cột trong dữ liệu thành hàng.
- `unstack`: thao tác này chuyển các hàng thành cột.

Chúng ta sẽ minh họa các thao tác này thông qua các ví dụ. Hãy xem xét một `DataFrame` nhỏ với các mảng chuỗi làm chỉ số hàng và cột:

```
data = pd.DataFrame(np.arange(6).reshape((2, 3)),
                    index=pd.Index(["Ohio", "Colorado"], name="state"),
                    columns=pd.Index(["one", "two", "three"], name="number"))
```

data

number	one	two	three
state			
Ohio	0	1	2
Colorado	3	4	5

```
number  one  two  three
state
Ohio    0    1    2
Colorado 3    4    5
```

Sử dụng phương thức `stack` trên dữ liệu này sẽ xoay các cột thành hàng, tạo ra một Series:

```
result = data.stack()
```

result

```
state    number
Ohio     one      0
         two      1
         three    2
Colorado one      3
         two      4
         three    5
dtype: int64
```

```
state    number
Ohio     one      0
         two      1
         three    2
Colorado one      3
         two      4
         three    5
dtype: int64
```

Từ một Series có chỉ số phân cấp, chúng ta có thể sắp xếp lại dữ liệu trở lại thành một DataFrame bằng `unstack`:

```
result.unstack()
```

number	one	two	three
state			
Ohio	0	1	2
Colorado	3	4	5

```
number    one  two  three
state
Ohio      0   1   2
Colorado  3   4   5
```

Khi sử dụng phương thức `unstack`, dữ liệu được tạo ra có thể có dữ liệu không quan sát được nếu không phải tất cả các giá trị đều được tìm thấy trong mỗi nhóm con:

```
s1 = pd.Series([0, 1, 2, 3], index=["a", "b", "c", "d"], dtype="Int64")
```

```
s2 = pd.Series([4, 5, 6], index=["c", "d", "e"], dtype="Int64")
```

```
data2 = pd.concat([s1, s2], keys=["one", "two"])
```

```
data2
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
two  c    4  
     d    5  
     e    6  
dtype: Int64
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
two  c    4  
     d    5  
     e    6  
dtype: Int64
```

Phương thức `stack` lọc bỏ dữ liệu không quan sát được theo mặc định, vì vậy thao tác này dễ dàng đảo ngược hơn:

```
data2.unstack()
```

	a	b	c	d	e
one	0	1	2	3	<NA>
two	<NA>	<NA>	4	5	6

```
      a    b  c  d  e  
one   0    1  2  3 <NA>  
two <NA> <NA> 4  5  6
```

```
data2.unstack().stack()
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
two  c    4  
     d    5  
     e    6  
dtype: Int64
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
two  c    4  
     d    5  
     e    6  
dtype: Int64
```

```
data2.unstack().stack(dropna=False)
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
     e  <NA>  
two  a  <NA>  
     b  <NA>  
     c    4  
     d    5  
     e    6  
dtype: Int64
```

```
one  a    0  
     b    1  
     c    2  
     d    3  
     e  <NA>  
two  a  <NA>  
     b  <NA>  
     c    4  
     d    5  
     e    6  
dtype: Int64
```

Khi `unstack` trong một DataFrame, cấp được `unstack` trở thành cấp thấp nhất trong kết quả:

```
df = pd.DataFrame({"left": result, "right": result + 5},  
                  columns=pd.Index(["left", "right"], name="side"))
```

df

	side	left	right
state	number		
Ohio	one	0	5
	two	1	6
	three	2	7
Colorado	one	3	8
	two	4	9
	three	5	10

```
side      left  right  
state  number  
Ohio   one     0     5  
       two     1     6  
       three   2     7  
Colorado one    3     8  
         two    4     9  
         three  5    10
```

```
df.unstack(level="state")
```

side	left		right	
state	Ohio	Colorado	Ohio	Colorado
number				
one	0	3	5	8
two	1	4	6	9
three	2	5	7	10

side	left		right	
state	Ohio	Colorado	Ohio	Colorado
number				
one	0	3	5	8
two	1	4	6	9
three	2	5	7	10

Tương tự như `unstack`, khi gọi `stack`, chúng ta có thể chỉ định tên của trục:

```
df.unstack(level="state").stack(level="side")
```

	state	Ohio	Colorado
number	side		
one	left	0	3
	right	5	8
two	left	1	4
	right	6	9
three	left	2	5
	right	7	10

state	Colorado	Ohio
number	side	
one	left	3 0
	right	8 5
two	left	4 1
	right	9 6
three	left	5 2
	right	10 7

9.3.2. Xoay trục từ định dạng dài sang rộng và ngược lại

Một cách phổ biến để lưu trữ nhiều chuỗi thời gian trong cùng một dữ liệu định dạng đôi khi được gọi là định dạng dài. Trong định dạng này, các giá trị riêng lẻ được biểu thị bằng một hàng duy nhất trong bảng thay vì nhiều giá trị trên mỗi hàng.

Hãy quan sát dữ liệu thường được định dạng kiểu dài như sau:

```
data = pd.read_csv("data/macrodta.csv")
```

```
data = data.loc[:, ["year", "quarter", "realgdp", "infl", "unemp"]]
```

```
data.head()
```

	year	quarter	realgdp	infl	unemp
0	1959	1	2710.349	0.00	5.8
1	1959	2	2778.801	2.34	5.1
2	1959	3	2775.488	2.74	5.3
3	1959	4	2785.204	0.27	5.6
4	1960	1	2847.699	2.31	5.2

	year	quarter	realgdp	infl	unemp
0	1959	1	2710.349	0.00	5.8
1	1959	2	2778.801	2.34	5.1
2	1959	3	2775.488	2.74	5.3
3	1959	4	2785.204	0.27	5.6
4	1960	1	2847.699	2.31	5.2

Chúng ta sử dụng `pandas.PeriodIndex` để kết hợp các cột `year` và `quarter` nhằm đặt chỉ mục bao gồm các giá trị datetime vào cuối mỗi quý:

```
periods = pd.PeriodIndex(year=data.pop("year"),
                        quarter=data.pop("quarter"),
                        name="date")
```

```
periods
```

```
PeriodIndex(['1959Q1', '1959Q2', '1959Q3', '1959Q4', '1960Q1', '2008Q3',
            '2008Q4', '2009Q1', '2009Q2', '2009Q3'],
            dtype='period[Q-DEC]', name='date')
```

```
data.index = periods.to_timestamp("D")
```

```
data.head()
```

	realgdp	infl	unemp
date			
1959-01-01	2710.349	0.00	5.8
1959-04-01	2778.801	2.34	5.1
1959-07-01	2775.488	2.74	5.3
1959-10-01	2785.204	0.27	5.6
1960-01-01	2847.699	2.31	5.2

```

      realgdp  infl  unemp
date
1959-01-01  2710.349  0.00   5.8
1959-04-01  2778.801  2.34   5.1
1959-07-01  2775.488  2.74   5.3
1959-10-01  2785.204  0.27   5.6
1960-01-01  2847.699  2.31   5.2

```

Chúng ta đã sử dụng phương thức `pop` trên DataFrame, phương thức này trả về một cột đồng thời xóa nó khỏi DataFrame.

Sau đó, chúng ta chọn một tập hợp con các cột và đặt tên cho chỉ số cột là `"item"`:

```
data = data.reindex(columns=["realgdp", "infl", "unemp"])
```

```
data.columns.name = "item"
```

```
data.head()
```

item	realgdp	infl	unemp
date			
1959-01-01	2710.349	0.00	5.8
1959-04-01	2778.801	2.34	5.1
1959-07-01	2775.488	2.74	5.3
1959-10-01	2785.204	0.27	5.6
1960-01-01	2847.699	2.31	5.2

```

item      realgdp  infl  unemp
date
1959-01-01  2710.349  0.00   5.8
1959-04-01  2778.801  2.34   5.1
1959-07-01  2775.488  2.74   5.3
1959-10-01  2785.204  0.27   5.6

```

Cuối cùng, dữ liệu được định dạng lại bằng phương thức `stack`, trong đó các cột bên trong được chuyển thành một cặp chỉ số mới. Tiếp theo, phương thức `reset_index` được sử dụng để

đưa hệ chỉ số phân cấp này trở lại dạng cột thông thường. Cột chứa giá trị dữ liệu sau đó được đặt tên rõ ràng là `"value"` nhằm đảm bảo tính minh bạch và nhất quán trong xử lý dữ liệu.

```
long_data = (data.stack()  
              .reset_index()  
              .rename(columns={0: "value"}))
```

```
long_data[:10]
```

	date	item	value
0	1959-01-01	realgdp	2710.349
1	1959-01-01	infl	0.000
2	1959-01-01	unemp	5.800
3	1959-04-01	realgdp	2778.801
4	1959-04-01	infl	2.340
5	1959-04-01	unemp	5.100
6	1959-07-01	realgdp	2775.488
7	1959-07-01	infl	2.740
8	1959-07-01	unemp	5.300
9	1959-10-01	realgdp	2785.204

Trong định dạng `long` dành cho dữ liệu chuỗi thời gian đa biến, mỗi hàng trong bảng biểu thị một quan sát duy nhất.

Dữ liệu thường được lưu trữ theo cách này trong các cơ sở dữ liệu quan hệ sử dụng SQL, vì lược đồ cố định (bao gồm tên cột và kiểu dữ liệu) cho phép số lượng giá trị phân biệt trong cột `"item"` có thể thay đổi linh hoạt khi dữ liệu mới được thêm vào bảng. Trong ví dụ trước, các cột `date` và `item` thường được sử dụng làm **khóa chính** (theo thuật ngữ của cơ sở dữ liệu quan hệ), nhằm đảm bảo tính toàn vẹn quan hệ và hỗ trợ các thao tác nối bảng (join) một cách hiệu quả.

Tuy nhiên, trong một số trường hợp, việc thao tác với dữ liệu ở định dạng này có thể kém trực quan hoặc khó xử lý hơn. Khi đó, bạn có thể muốn chuyển dữ liệu sang định dạng bảng rộng

(*wide format*), trong đó mỗi mục (`item`) sẽ trở thành một cột riêng, và bảng được lập chỉ số theo thời gian (`date`). Phương thức `pivot` của `DataFrame` trong Pandas cho phép thực hiện chính xác quá trình chuyển đổi này.

```
pivoted = long_data.pivot(index="date", columns="item",
                           values="value")
```

```
pivoted.head()
```

item	infl	realgdp	unemp
date			
1959-01-01	0.00	2710.349	5.8
1959-04-01	2.34	2778.801	5.1
1959-07-01	2.74	2775.488	5.3
1959-10-01	0.27	2785.204	5.6
1960-01-01	2.31	2847.699	5.2

Hai tham số đầu tiên được truyền vào phương thức `pivot` lần lượt xác định các cột sẽ được sử dụng làm chỉ số hàng (*row index*) và chỉ số cột (*column index*). Tham số thứ ba, không bắt buộc, xác định cột chứa giá trị sẽ được sử dụng để lấp đầy bảng `DataFrame` sau khi chuyển đổi.

Giả sử bạn có hai cột giá trị riêng biệt và mong muốn thay đổi cấu trúc của chúng đồng thời (tức là chuyển đổi từ định dạng "long" sang "wide" cho nhiều cột cùng lúc). Trong trường hợp này, `pivot` không thể trực tiếp xử lý nhiều cột giá trị trong một lệnh duy nhất. Bạn sẽ cần thực hiện thao tác này bằng cách sử dụng các phương thức khác như `pivot_table`, hoặc áp dụng `pivot` cho từng cột một, sau đó kết hợp kết quả lại.

```
long_data["value2"] = np.random.standard_normal(len(long_data))
long_data[:10]
```

	date	item	value	value2
0	1959-01-01	realgdp	2710.349	-0.327235
1	1959-01-01	infl	0.000	-0.230390
2	1959-01-01	unemp	5.800	-0.731397
3	1959-04-01	realgdp	2778.801	-1.294915
4	1959-04-01	infl	2.340	-0.400165
5	1959-04-01	unemp	5.100	-1.425981
6	1959-07-01	realgdp	2775.488	0.553444
7	1959-07-01	infl	2.740	-0.239593
8	1959-07-01	unemp	5.300	-0.843350
9	1959-10-01	realgdp	2785.204	0.035293

Khi không chỉ định tham số cuối cùng (tức là cột giá trị) trong phương thức `pivot`, kết quả trả về là một `DataFrame` có cấu trúc **cột phân cấp**. Trong cấu trúc này, các giá trị duy nhất từ cột ban đầu, thường là cột chứa nhiều biến giá trị, sẽ được đưa lên cấp thứ hai của cột trong bảng kết quả, trong khi cấp đầu tiên được xác định bởi cột chỉ mục cột đã chỉ định trong `pivot`.

Kiểu tổ chức dữ liệu này rất hữu ích khi bạn muốn giữ nguyên nhiều cột giá trị đồng thời mà không làm mất thông tin, và thường xuất hiện trong các phân tích bảng chéo hoặc tổng hợp dữ liệu đa chiều.

Hãy cùng quan sát ví dụ minh họa để hiểu rõ hơn cách thức hoạt động của cấu trúc cột phân cấp trong Pandas.

```
pivoted = long_data.pivot(index="date", columns="item")
pivoted.head()
```

	value			value2		
item	infl	realgdp	unemp	infl	realgdp	unemp
date						
1959-01-01	0.00	2710.349	5.8	-0.230390	-0.327235	-0.731397
1959-04-01	2.34	2778.801	5.1	-0.400165	-1.294915	-1.425981
1959-07-01	2.74	2775.488	5.3	-0.239593	0.553444	-0.843350
1959-10-01	0.27	2785.204	5.6	0.046383	0.035293	-0.537971
1960-01-01	2.31	2847.699	5.2	-1.295712	-0.478144	0.061955

```
pivoted["value"].head()
```

item	infl	realgdp	unemp
date			
1959-01-01	0.00	2710.349	5.8
1959-04-01	2.34	2778.801	5.1
1959-07-01	2.74	2775.488	5.3
1959-10-01	0.27	2785.204	5.6
1960-01-01	2.31	2847.699	5.2

Cần lưu ý rằng phương thức `pivot` trong Pandas thực chất tương đương với việc tạo một hệ chỉ số phân cấp (*hierarchical index*) bằng cách sử dụng `set_index`, sau đó áp dụng phương thức `unstack`.

Cụ thể, `set_index` cho phép xác định một hoặc nhiều cột làm chỉ số cho bảng dữ liệu, từ đó cấu trúc lại bảng theo hướng đa chiều. Phương thức `unstack` tiếp theo sẽ “xoay” một trong các cấp chỉ số đó thành cột, tạo ra một bảng mới theo định dạng “wide”.

Cách tiếp cận này đặc biệt hữu ích khi bạn cần kiểm soát linh hoạt quá trình chuyển đổi dữ liệu, hoặc khi muốn áp dụng các thao tác tương tự `pivot` trong các tình huống không thể sử dụng trực tiếp `pivot`.

Ví dụ cụ thể dưới đây sẽ minh họa rõ ràng mối quan hệ tương đương giữa `pivot` và tổ hợp `set_index` + `unstack`.

```
unstacked = long_data.set_index(["date", "item"]).unstack(level="item")
```

```
unstacked.head()
```

	value			value2		
item	infl	realgdp	unemp	infl	realgdp	unemp
date						
1959-01-01	0.00	2710.349	5.8	-0.230390	-0.327235	-0.731397
1959-04-01	2.34	2778.801	5.1	-0.400165	-1.294915	-1.425981
1959-07-01	2.74	2775.488	5.3	-0.239593	0.553444	-0.843350
1959-10-01	0.27	2785.204	5.6	0.046383	0.035293	-0.537971
1960-01-01	2.31	2847.699	5.2	-1.295712	-0.478144	0.061955

Một thao tác ngược lại với phương thức `pivot` trong `DataFrame` là phương thức `pandas.melt`. Thay vì chuyển một cột thành nhiều cột trong một bảng dữ liệu mới như `pivot`, `melt` thực hiện việc **gộp nhiều cột thành một**, từ đó tạo ra một `DataFrame` có **số hàng lớn hơn** so với đầu vào — tức là định dạng “long”.

Phép biến đổi này thường được sử dụng để đưa dữ liệu về **dạng chuẩn hóa** (*normalized form*), phù hợp với các quy tắc của cơ sở dữ liệu quan hệ và thuận tiện cho nhiều thao tác phân tích, trực quan hóa hoặc lưu trữ.

Hãy cùng xem một ví dụ minh họa cho quá trình này:

```
df = pd.DataFrame({"key": ["foo", "bar", "baz"],
                   "A": [1, 2, 3],
                   "B": [4, 5, 6],
                   "C": [7, 8, 9]})
df
```

	key	A	B	C
0	foo	1	4	7
1	bar	2	5	8
2	baz	3	6	9

Trong nhiều tình huống, cột `"key"` đóng vai trò là một **chỉ báo nhóm**, trong khi các cột còn lại chứa các giá trị dữ liệu cần được gộp lại. Khi sử dụng phương thức `pandas.melt`, chúng ta cần chỉ rõ những cột nào (nếu có) sẽ được giữ lại làm chỉ báo nhóm.

Cụ thể, trong ví dụ này, chúng ta sử dụng `"key"` làm **chỉ báo nhóm duy nhất**, và tiến hành gộp các cột còn lại thành một cột duy nhất chứa giá trị, đi kèm với một cột khác ghi tên thuộc tính ban đầu. Đây là cách làm phổ biến để chuyển dữ liệu từ định dạng "wide" sang "long" mà vẫn bảo toàn thông tin nhóm.

Ví dụ dưới đây sẽ minh họa rõ cách sử dụng `melt` với `"key"` làm chỉ báo nhóm.

```
melted = pd.melt(df, id_vars="key")
```

```
melted
```

	key	variable	value
0	foo	A	1
1	bar	A	2
2	baz	A	3
3	foo	B	4
4	bar	B	5
5	baz	B	6
6	foo	C	7
7	bar	C	8
8	baz	C	9

Sau khi đã chuyển đổi dữ liệu sang định dạng “long” bằng phương thức `melt`, chúng ta hoàn toàn có thể **khôi phục lại bố cục ban đầu** bằng cách sử dụng phương thức `pivot`. Phép biến đổi này cho phép tái cấu trúc bảng dữ liệu, trong đó một cột sẽ được sử dụng làm chỉ mục hàng, một cột khác làm tên cột mới, và cột cuối cùng chứa giá trị dữ liệu.

Việc sử dụng `pivot` trong trường hợp này giúp minh họa rõ ràng tính **tương hỗ giữa `melt` và `pivot`**, qua đó cho phép linh hoạt chuyển đổi qua lại giữa hai định dạng dữ liệu — một kỹ năng quan trọng trong xử lý và phân tích dữ liệu bằng Pandas.

Sau đây là ví dụ cụ thể cho quá trình chuyển dữ liệu về định dạng ban đầu bằng `pivot`.

```
reshaped = melted.pivot(index="key", columns="variable",  
                          values="value")  
reshaped
```

variable	A	B	C
key			
bar	2	5	8
baz	3	6	9
foo	1	4	7

Kết quả của phép biến đổi bằng `pivot` thường tạo ra một **chỉ số mới** dựa trên cột được sử dụng làm tên hàng. Trong một số trường hợp, chúng ta có thể muốn chuyển chỉ số này trở lại thành cột dữ liệu thông thường, đặc biệt nếu cần tiếp tục xử lý hoặc hiển thị dữ liệu ở định dạng bảng phẳng.

Phương thức `reset_index` được sử dụng để thực hiện thao tác này. Cụ thể, `reset_index` đưa các cấp của chỉ số về thành cột của `DataFrame`, từ đó giúp đơn giản hóa cấu trúc bảng và hỗ trợ tốt hơn cho các bước xử lý tiếp theo.

Ví dụ dưới đây sẽ minh họa rõ cách sử dụng `reset_index` sau khi đã thực hiện `pivot`.

```
reshaped.reset_index()
```

variable	key	A	B	C
0	bar	2	5	8
1	baz	3	6	9
2	foo	1	4	7

```
pd.melt(df, id_vars="key", value_vars=["A", "B"])
```

	key	variable	value
0	foo	A	1
1	bar	A	2
2	baz	A	3
3	foo	B	4
4	bar	B	5
5	baz	B	6

```
pd.melt(df, value_vars=["A", "B", "C"])
```

	variable	value
0	A	1
1	A	2
2	A	3
3	B	4
4	B	5
5	B	6
6	C	7
7	C	8
8	C	9

Sau khi đã nắm vững một số kiến thức cơ bản về Pandas, bao gồm nhập dữ liệu, làm sạch và tổ chức lại cấu trúc bảng dữ liệu, chúng ta có thể chuyển sang nội dung trực quan hóa dữ liệu với thư viện Matplotlib.

Việc trực quan hóa dữ liệu đóng vai trò quan trọng trong việc hiểu rõ xu hướng, mối quan hệ và các đặc điểm ẩn chứa trong tập dữ liệu. Đây cũng là công cụ hỗ trợ ra quyết định hiệu quả trong nhiều lĩnh vực ứng dụng thực tiễn.

Chúng ta sẽ quay lại khám phá thêm nhiều tính năng nâng cao khác của Pandas ở các chương sau, khi bước vào các phân tích chuyên sâu và kỹ thuật xử lý dữ liệu nâng cao hơn.